

E-COMMERCE INTELLIGENCE PLATFORM

A Comprehensive Guide to Analytics Engineering & Machine Learning

Case Study: Ingesting and Mining 33.8 Million Instacart Transactions

Target Competencies Developed:

- Analytics Engineering & Dimensional Star Schema Modeling (DuckDB / SQL)
- High-Performance Vectorized ETL Pipelines (Polars / Parquet)
- Data-Driven Customer Clustered RFM Segmentations (Scikit-Learn K-Means)
- Machine Learning forecasting, Association Basket Rules, & Collaborative Recommenders

Date: June 2026

Platform Version: 1.0 (Production-Grade Fallback Active)

1. Medallion Data Architecture (Bronze -> Silver -> Gold)

To manage and clean the 33.8 million transaction lines, the platform utilizes a Medallion Architecture. This design divides the analytical pipeline into progressive quality tiers to build a reliable data product.

Bronze Layer: Ingestion & Serialization

- * **Ingestion Source:** Copies raw CSV datasets (700MB uncompressed) from the local archive folder.
- * **Parquet Conversion:** Converts rows to Snappy-compressed Parquet files, reducing space on disk to under 150MB.
- * **Why Parquet over CSV?:** CSVs are row-oriented and stored as text. Parquet is columnar (allowing fast column-only reads) and typed, saving disk size and memory when querying millions of items.
- * **Polars vs. Pandas:** Pandas runs on a single thread and creates massive overhead. Polars is built in Rust and uses all CPU cores concurrently to stream data in parallel.

Silver Layer: Cleaning & Schema Validation

- * **Data Hygiene:** Enforces database column types, handles null values (setting first order days-since-prior to 0), and screens for negative metrics.
- * **Temporal Calibration:** Since the raw Instacart data lacks absolute calendar timestamps, we anchored each user's starting point to 2025 and added cumulative days per order to build a realistic calendar timeline.

Gold Layer: Star Schema Modeling

In the final layer, cleaned files are loaded into a localized DuckDB relational database, organizing them into a structured Star Schema optimized for downstream querying and BI dashboarding.

2. Dimensional Modeling & Star Schema

For the database, we modeled the relations as a Star Schema. Fact tables capture numerical metrics, and Dimension tables capture descriptive contexts. This layout optimizes search performance and reduces duplication.

- * **FactOrders (Fact Table):** Holds the numeric transactions of every items. Fields: order_id, user_id (FK), product_id (FK), date_key (FK), add_to_cart_order, quantity (1), unit_price, subtotal.

- * **DimCustomers (Dimension):** Customer lifetime details. Fields: user_id, first_order_date, total_orders, total_spend, rfm_segment, clv, kmeans_segment.

- * **DimProducts (Dimension):** Items inventory catalog. Fields: product_id, product_name, aisle_id (FK), department_id (FK), unit_price, abc_class.

- * **DimDate (Dimension):** Temporal calendar granularity at the date-hour key (YYYYMMDDHH) level. Fields: date_key, date, day_of_week, hour_of_day, month, year.

DuckDB: The Analytical Database Engine

SQLite is a row-oriented transactional (OLTP) engine meant for simple lookups. In contrast, DuckDB is a vectorized, columnar engine designed for analytical (OLAP) queries. It stores columns contiguously in memory and processes vector arrays of values. This allows DuckDB to compute sums, averages, and group-bys on 33.8 million rows in milliseconds.

Performance Indexing

We created indexes on foreign key dimensions (user_id, product_id, date_key) inside the fact_orders table. When joining tables, DuckDB references these pre-sorted indexes, avoiding full table scans.

3. Semantic KPI Views

To abstract complexity, we built a Semantic Layer using SQL Views inside DuckDB. A SQL View is a stored query rather than a physical table. Whenever a view is queried, DuckDB dynamically computes the aggregation. This ensures that the Streamlit app always pulls consistent, real-time calculations directly from the warehouse.

- * **v_executive_kpis:** Calculates baseline KPIs: Total Revenue, Total Orders, Average Order Value (AOV), Active Customers, and Repeat Purchase Rate.

- * **v_product_performance:** Calculates total sales counts and compiles Pareto cumulative revenues to categorize inventory classes.

4. Customer Segmentation (RFM + K-Means)

Rule-Based RFM Metrics

Customers are scored on a 1-5 scale across: Recency (days since last order), Frequency (total orders), and Monetary value (total spent). High-performing scoring yields segments like 'Champions' (high recency/frequency/monetary) and 'At Risk' (historically valuable but inactive).

Data-Driven K-Means Clustering

To identify customer groupings mathematically, we train a K-Means algorithm:

- * **Log Transform:** RFM values are heavily skewed (a few shoppers spend thousands while most spend under \$100). Log scaling (`np.log1p`) normalizes right-skewed tails.

- * **StandardScaler:** Since K-Means relies on distance calculations, we standardize values. This prevents Monetary (dollars) from dominating Recency (days) in scaling.

- * **Cluster Profiling:** The model forms 4 segments, which we automatically label based on their relative spend and frequency averages: 'VIPs', 'At Risk', 'Lost', and 'New Customers'.

5. Daily Demand Forecasting

To optimize inventory planning, we forecast future revenue using statistical time-series and machine learning regressor models.

Holt-Winters Exponential Smoothing

Holt-Winters models baseline, trend, and weekly cycles. In simulated datasets, timelines gradually tail off. If we use a standard linear trend, it projects revenue downward to infinity. We configured a Damped Trend (`damped_trend=True`) and clipped future values (`np.clip(..., 0, None)`) to ensure forecasts stabilize realistically at a non-negative baseline.

RandomForest Regressor (XGBoost Fallback)

Machine learning models cannot predict time-series without feature engineering. We create:

- * **Lag Features:** Sales from yesterday (`lag_1`), a week ago (`lag_7`), and two weeks ago (`lag_14`) to capture baseline offsets.
- * **Rolling Windows:** 7-day and 30-day rolling averages to capture medium-term business momentum.
- * **Cross-Platform Fallback:** XGBoost requires compilation libraries (OpenMP) missing by default on macOS. We catch load errors and automatically substitute Scikit-learn's RandomForestRegressor, yielding robust execution.

6. Transaction Anomaly Detection

We use an Isolation Forest algorithm on daily order count and revenue. It isolates outliers by building tree partitions. Normal days require many splits to isolate, while anomalous days (extreme peak sales or drop-offs) require very few splits. We configured a contamination rate of 1% (`contamination=0.01`) to flag the top operational anomalies in the history.

7. Product Recommendation Systems

The platform utilizes two recommendation frameworks to personalize checkout experiences and drive average order value.

Market Basket Analysis (MBA)

Mines co-purchase relationships inside transaction carts based on association rules:

- * **Support:** The fraction of total orders containing both Product A and Product B.
- * **Confidence:** The conditional probability that a customer buys Product B given they have placed Product A in their cart.
- * **Lift:** The ratio of observed joint purchase to independent probability. $Lift > 1$ indicates that A and B are bought together far more frequently than by random chance.

We executed a high-performance Polars self-join on 22.3 million row item combinations, mining rules (e.g. Fuji Apples + Organic Strawberries has a 67.0x Lift).

Item-Item Collaborative Filtering

Pre-computes personalized suggestions for customers who have historical order patterns:

- * **Sparse Matrix:** Constructs a scipy sparse category matrix of user-product purchase counts, saving memory.
- * **Cosine Similarity:** Computes similarity angles between product columns to build co-occurrence similarity maps.
- * **Scoring:** Recommends candidate items by multiplying a user's purchase counts against the item-to-item similarity matrix (filtering out already purchased items).

8. Executive Insights & Decision Frameworks

To ensure the system is valuable out-of-the-box, we compiled a rich, deterministic executive report that executes queries on Gold semantic views. Here are the core insights and operational actions:

- * **Customer Pareto Split:** The Champions RFM segment contributes 62.9% of total revenue while representing only 26.5% of the customer base, proving the 80/20 rule. Action: Prioritize VIP retention campaigns.
- * **Inventory ABC Split:** Class A products constitute 9.8% of the catalog but drive 80.0% of total revenue. Class C is 68.7% of the catalog driving just 5.0% of sales. Action: Focus quality checks on Class A, and move Class C to just-in-time logistics.
- * **Logistics Peak Windows:** Peak transactions occur on Wednesday at 11:00. Action: Staff logistics 2 hours ahead of this peak window.

9. Production Best Practices (Do's & Don'ts)

What to Do

- * **Use Vectorized ETL:** Always use multi-threaded columnar frameworks (Polars/Parquet) when processing datasets over 10M rows to avoid memory overhead.
- * **Damp Trends:** Always damp and clip forecasting trends on finite/staggered simulations so they don't project unrealistic values.
- * **Indexes:** Create primary/foreign key indexes on analytical tables (DuckDB) to speed up transactional join queries.

What Not to Do

- * **Don't Loop:** Never use Python loops (for-loops) to clean or transform data rows. Vectorized operations run 100x faster.
- * **Don't use CSV for OLAP:** Never query raw text CSVs inside dashboard loops. Load tables into an indexed database (DuckDB).